

SmartParking

Documentazione del progetto – Testing e Verifica del Software

Francesco Tomasoni

A.A. 2025/2026

Contents

1	Introduzione e requisiti	3
1.1	Descrizione del sistema	3
1.2	Requisiti funzionali	3
1.3	Requisiti non funzionali	4
1.4	Macchina a stati	4
1.5	Glossario	4
2	Modello Asmeta	5
2.1	Il modello (<code>asmeta/src/SmartParking.asm</code>)	5
2.1.1	Il modello semplificato per il model checking	6
2.1.2	Simulazione manuale	6
2.2	Scenari Avalla	7
2.3	AsmetaV – validazione con copertura (Vc)	8
2.4	Model checking (<code>asmeta/src/SmartParking_MC.asm</code>)	9
2.4.1	Model Advisor (opzionale)	10
2.4.2	ATGT – generazione automatica di scenari (opzionale)	11
3	Implementazione Java	14
3.1	Struttura del progetto Maven	14
3.2	La macchina a stati	14
3.3	Interfaccia utente (Vaadin)	14
3.4	Test end-to-end con Selenium	16
4	Testing Java	17
4.1	Struttura dei test	17
4.2	MC/DC	18
4.3	Combinatorial testing (pairwise)	18
4.4	Mockito	19
4.5	Model Based Testing	19
4.6	Esecuzione e copertura	19
5	JML e verifica statica (ESC)	20
5.1	Invarianti	21
5.2	Costruttori	21
5.3	Metodo <code>assegnaPosto</code>	22
5.4	Metodo <code>liberaPosto</code>	22
5.5	Metodi <code>getter</code>	23
5.6	Verifica dei contratti con OpenJML (ESC)	23

6	Analisi statica e ispezione del codice con LLM	24
6.1	Checklist preliminare	25
6.2	Le osservazioni e come le ho gestite	25
6.3	Bilancio	26
7	Continuous Integration	26
8	Conclusioni	27
8.1	Riepilogo delle evidenze	27
8.2	Cosa mi porto a casa	27

1 Introduzione e requisiti

1.1 Descrizione del sistema

Il sistema che ho scelto di fare è un parcheggio automatico. Ci sono due sbarre, una in ingresso e una in uscita, dei sensori che si accorgono quando arriva un'auto e un totem che legge che tipo di utente è. I posti sono di due tipi: standard e riservati ai disabili. Nel modello ne ho messo uno solo per tipo, altrimenti il model checker dopo esplode.

Gli utenti sono tre:

- **STD**: l'utente occasionale, è l'unico che paga;
- **DISABILE**: ha il permesso, non paga e ha la precedenza sui posti riservati;
- **ABBONATO**: ha l'abbonamento mensile e non paga.

Quando arriva un'auto il sistema la rileva, guarda che utente è e controlla se c'è un posto adatto. Se c'è alza la sbarra, se non c'è nega l'accesso.

Il caso un po' più interessante è quello del disabile. Lui i posti riservati li prende per primo, ma se sono finiti il sistema non lo respinge: gli dà un posto standard, se ce n'è uno libero. Questo l'ho chiamato *fallback* e torna spesso nel resto del documento, perché è il ramo che ho dovuto coprire con più attenzione un po' ovunque. Solo se sono finiti anche i posti standard allora viene negato l'accesso anche a lui.

In uscita paga solo l'utente STD, gli altri due escono e basta.

Il giro che ho fatto è questo: prima ho scritto il comportamento come macchina a stati in Asmeta, poi l'ho validato con gli scenari Avalla e con il model checking, e alla fine l'ho implementato in Java mettendo i contratti JML sul nucleo.

1.2 Requisiti funzionali

ID	Descrizione	Priorità
RF1	Rilevare l'arrivo di un veicolo in ingresso (sens_in) e passare da IDLE a CHK_IN.	Alta
RF2	Identificare il tipo di utente (STD, DISABILE, ABBONATO) e passare allo stato VER.	Alta
RF3	Per STD/ABBONATO, assegnare un posto standard se posti_std > 0 e aprire la sbarra (INGR).	Alta
RF4	Per DISABILE, assegnare in via prioritaria un posto disabile se posti_dis > 0.	Alta
RF5	Se per un DISABILE i posti disabili sono esauriti, tentare il fallback su posto standard.	Alta
RF6	Se nessun posto è disponibile (inclusi i fallback), negare l'accesso (NEG).	Alta
RF7	Da NEG, tornare in IDLE solo dopo che il veicolo si è allontanato (auto_via).	Media
RF8	Al transito in ingresso, decrementare il contatore del posto assegnato e tornare in IDLE.	Alta
RF9	Rilevare l'arrivo di un veicolo in uscita (sens_out) e passare a CHK_OUT.	Alta
RF10	In uscita, per STD richiedere il pagamento (TARIF) prima della sbarra.	Alta

ID	Descrizione	Priorità
RF11	In uscita, per DISABILE/ABBONATO saltare il pagamento e andare direttamente a USC.	Alta
RF12	Restare in TARIF finché il pagamento non è confermato.	Alta
RF13	Al transito in uscita, incrementare il contatore, azzerare la memoria del posto e tornare in IDLE.	Alta
RF14	Mantenere in memoria il tipo di posto occupato dall'ingresso fino all'uscita.	Media

1.3 Requisiti non funzionali

ID	Descrizione
RNF1	Sicurezza dei contatori: <code>posti_std</code> e <code>posti_dis</code> non devono mai essere negativi né superare la capacità iniziale, in ogni stato raggiungibile (verificato via model checking).
RNF2	Determinismo: a parità di stato e input monitorati, il sistema produce sempre lo stesso stato successivo.
RNF3	Assenza di stati bloccanti: da ogni stato transitorio esiste sempre un cammino che riporta il sistema in IDLE.

1.4 Macchina a stati

Qui sotto ci sono gli 8 stati del modello e le transizioni che li collegano. Il giro sopra è quello di ingresso, quello sotto è l'uscita, e da IDLE si parte sempre.

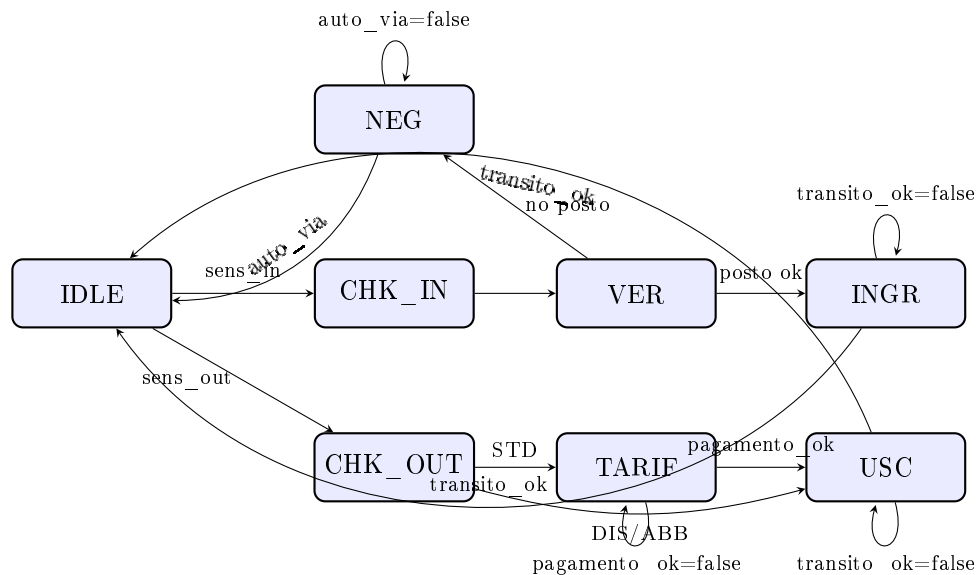


Figure 1: Diagramma della macchina a stati SmartParking (8 stati).

1.5 Glossario

Termine		Significato
Sensore ingresso/uscita	di	Rileva la presenza di un veicolo davanti a una sbarra.
Sbarra		Barriera fisica aperta solo in INGR/USC quando <code>transito_ok=true</code> .
Transito		Il veicolo è passato fisicamente sotto la sbarra aperta.
Tipo utente		Categoria del veicolo: STD, DISABILE o ABBONATO.
Posto standard / disabile		Le due categorie di posto auto.
Fallback		Politica per cui un DISABILE senza posti riservati riceve un posto standard.
Tariffa (TARIF)		Fase di pagamento, obbligatoria solo per STD in uscita.

2 Modello Asmeta

2.1 Il modello (asmeta/src/SmartParking.asm)

Partiamo dalla signature. Ho definito tre domini enumerativi: `StatoSistema` con gli 8 stati, `TipoUtente` con i tre tipi di utente e `TipoPosto` con i due tipi di posto più il valore `NESSUNO`, che uso quando non c'è nessun posto occupato.

Poi ci sono sei funzioni monitorate, che sono quelle che arrivano dall'esterno e che il modello subisce: `sens_in` e `sens_out` per i due sensori, `transito_ok` per dire che l'auto è passata sotto la sbarra, `auto_via` per dire che se n'è andata dopo un rifiuto, `utente_rilevato` per il tipo di utente letto dal totem e `pagamento_ok` per l'esito del pagamento.

Le funzioni controllate, cioè la memoria del sistema, sono quattro: `stato`, i due contatori `posti_std` e `posti_dis`, e `posto_assegnato`, che si ricorda che tipo di posto ha preso l'auto entrata. Quest'ultima serve perché in uscita devo sapere quale contatore rimettere a posto.

Poi ho aggiunto due funzioni derivate. La prima è `assegnabile`, ed è quella che mi interessava di più perché è n-aria: prende due argomenti, un utente e un posto, e dice se quel posto si può dare a quell'utente.

Listing 1: Funzione derivata assegnabile

```

1 derived assegnabile: Prod(TipoUtente, TipoPosto) -> Boolean
2
3 function assegnabile($u in TipoUtente, $p in TipoPosto) =
4     ($p = POSTO_STD and posti_std > 0) or
5     ($p = POSTO_DIS and $u = DISABILE and posti_dis > 0)

```

Letta a parole: un posto standard lo do a chiunque, basta che ce ne sia uno libero; un posto disabile lo do solo a un disabile, e solo se ce n'è uno libero. Il vantaggio di averla messa qui è che tutta la politica di assegnazione sta in due righe, in un punto solo. Essendo derivata non occupa memoria, viene ricalcolata a ogni stato partendo dai contatori.

La seconda derivata è `parcheggioPieno`, e l'ho scritta soprattutto per usare il `forall`:

Listing 2: Funzione derivata parcheggioPieno (uso di forall)

```

1 function parcheggioPieno =
2     (forall $u in TipoUtente with
3         (not assegnabile($u, POSTO_STD) and not assegnabile($u, POSTO_DIS)))

```

Diventa vera quando *nessun* tipo di utente può più entrare, né su un posto standard né su uno disabili. La controllo nello scenario `test_pieno_totale.avalla`: all'inizio è falsa perché c'è ancora posto, alla fine diventa vera.

Veniamo alle regole. Quella che conta davvero è `r_gestione_VER`, dove si decide chi entra e chi no. Qui ho usato una regola `let` per non ripetere `utente_rilevato` ogni volta, e dentro ho messo la derivata di prima:

Listing 3: Regola `r_gestione_VER` (assegnazione posto, con `let` e derivata)

```

1 rule r_gestione_VER =
2   let ($u = utente_rilevato) in
3     if assegnabile($u, POSTO_DIS) then
4       par
5         stato := INGR
6         posto_assegnato := POSTO_DIS
7       endpar
8     else
9       if assegnabile($u, POSTO_STD) then
10        par
11          stato := INGR
12          posto_assegnato := POSTO_STD
13        endpar
14      else
15        stato := NEG
16      endif
17    endif
18  endlet

```

L'ordine dei due `if` non è casuale. Provo prima con il posto disabili, che è vero solo se l'utente è disabili e c'è un riservato libero. Se quel test fallisce provo con il posto standard, e questo secondo ramo copre due casi diversi in una volta: l'ingresso normale di uno STD o di un abbonato, e il fallback del disabili quando i riservati sono finiti. Se falliscono tutti e due mando il sistema in NEG.

Le altre regole sono più semplici. `r_gestione_IDLE` guarda quale dei due sensori si è acceso e va in `CHK_IN` o in `CHK_OUT`. `r_gestione_CHK_IN` passa subito a VER, serve solo a rappresentare il tempo di lettura del totem. `r_gestione_INGR` aspetta il transito e poi scala il contatore giusto, guardando `posto_assegnato`. `r_gestione_CHK_OUT` manda in TARIF solo se l'utente è STD, altrimenti va dritto in USC. `r_gestione_TARIF` resta ferma finché il pagamento non va a buon fine. `r_gestione_USC` rimette a posto il contatore e azzera `posto_assegnato`.

2.1.1 Il modello semplificato per il model checking

Per il model checking non sono riuscito a usare il modello principale, quindi ne ho fatto una copia semplificata in `SmartParking_MC.asm`. Le differenze sono due.

La prima è che ho tolto le funzioni derivate e ho riscritto gli `if` in forma “appiattita”. Si comportano allo stesso modo, ma la traduzione verso NuSMV va liscia.

La seconda riguarda i contatori: nel modello principale sono `Integer`, e AsmetaSMV i domini infiniti non li accetta. Li ho quindi messi su un dominio finito `Posti = {0,1,2}`. Il 2 l'ho lasciato apposta anche se non serve: se avessi scritto `{0,1}` allora le proprietà `ag(posti_std <= 1)` e `ag(posti_dis <= 1)` sarebbero state vere per forza, solo per come è fatto il tipo, e non avrebbero verificato niente. Lasciando dentro il 2 il model checker deve davvero dimostrare che quel valore non viene mai raggiunto.

2.1.2 Simulazione manuale

La prima cosa che ho fatto, appena scritto il modello, è stata aprirlo nel simulatore e fare qualche step a mano, per vedere se si comportava come mi aspettavo prima di mettermi a scrivere gli

scenari.

	Type	Functions	^	State 0	State 1	State 2	State 3	State 4	State 5	
	M	sens_in		false	true	true	true	true		
	M	sens_out		false	false	false	false	false		
	C	stato		IDLE	IDLE	CHK_IN VER	INGR	IDLE		
	M	utente_rilevato					STD	STD		
	C	posti_std					1	1	0	
	C	posti_dis					1	1	1	
	C	posto_assegnato						POSTO_STD	POSTO_STD	
	M	transito_ok						true		

Figure 2: Simulatore – ingresso STD completo

Nello screenshot c'è il giro completo di un ingresso STD: metto **sens_in** a true e passo in **CHK_IN**, poi imposto l'utente e la macchina va in **VER**, da lì in **INGR** perché c'è posto, e infine con **transito_ok** torno in **IDLE** con **posti_std** sceso da 1 a 0. Una cosa che all'inizio mi aveva confuso: l'animatore mostra una funzione solo a partire dallo stato in cui viene letta o scritta la prima volta, quindi nelle prime colonne certe righe sembrano mancare, ma è normale.

	Type	Functions	^	State 0	State 1	State 2	State 3	State 4	State 5	State 6	State 7	State 8	State 9	State 10
	M	sens_in		false	true	true	true	true	false	false	false	false	false	
	M	sens_out		false	false	false	false	false	true	true	true	true	true	
	C	stato		IDLE	IDLE	CHK_IN VER	INGR	IDLE	CHK_OUT	TARIF	TARIF	USC	IDLE	
	M	utente_rilevato				STD	STD	STD	STD	STD	STD	STD	STD	
	C	posti_std				1	1	0	0	0	0	0	1	
	C	posti_dis				1	1	1	1	1	1	1	1	
	C	posto_assegnato					POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	NESSUNO
	M	transito_ok					true	true	true	true	true	true	true	
	M	pagamento_ok								false	true	true		

Figure 3: Simulatore – uscita STD con pagamento

Qui invece c'è l'uscita, sempre nella stessa sessione. Si vede il passaggio per **TARIF**, che è la parte che riguarda solo gli STD, e alla fine **posti_std** torna a 1 e **posto_assegnato** torna a **NESSUNO**.

2.2 Scenari Avalla

Gli scenari che ho scritto sono questi:

Scenario	Cosa prova
test_abbonato.avalla	Giro completo di un ABBONATO: entra, e in uscita salta il pagamento.
test_disabile.avalla	Giro completo di un DISABILE su posto riservato; controllo che <code>posto_assegnato</code> resti <code>POSTO_DIS</code> per tutta la sosta e si azzeri solo in uscita.
test_pieno.avalla	Uno STD prende l'unico posto, un secondo STD viene respinto.
test_disabile_- fallback.avalla	Il fallback: due disabili di fila, il secondo trova i riservati finiti e prende un posto standard.
test_pagamento_- std.avalla	Uscita STD con un pagamento che va male (resta in <code>TARIF</code>) e poi uno che va bene.
test_pieno_- totale.avalla	Parcheggio pieno del tutto (uno STD più un disabile), il terzo utente viene respinto.
test_random_- 30steps.avalla	Generato in automatico: 30 step random fatti nel simulatore ed esportati con "export to avalla".

Il primo, `test_abbonato.avalla`, era già dato ma non funzionava. Aveva due errori: inizializzava `posti_std` a 5, mentre nel modello parto da 1, e controllava uno stato chiamato `PARK` che nel mio modello non esiste, visto che si chiama `IDLE`. Corretti quei due punti gira senza problemi.

Type	Functions	State 4	State 5	State 6	State 7	State 8	State 9	State 10	State 11	State 12	State 13
C	step__	4	5	6	7	8	8	8	8	8	8
C	stato	IDLE	CHK_OUT	USC	IDLE	IDLE	IDLE	IDLE	IDLE	IDLE	IDLE
C	sens_in	false	false	false	false	false	false	false	false	false	false
C	result	1	1	1	1	1	1	1	1	1	1
C	utente_rilevato	ATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO
C	posto_assegnato	_STD	POSTO_STD	POSTO_STD	POSTO_STD	NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO
C	transito_ok	false	false	true	true	true	true	true	true	true	true
C	posti_std	0	0	0	1	1	1	1	1	1	1
C	sens_out	true	false	false	false	false	false	false	false	false	false

Figure 4: Animazione di `test_abbonato.avalla`

Nell'animazione dell'abbonato la cosa da guardare è la fase di uscita: da `CHK_OUT` si salta direttamente a `USC`, `TARIF` non compare proprio. È esattamente la differenza tra abbonato e utente `STD`.

Qui invece si vedono i due disabili uno dietro l'altro: il primo prende il posto riservato, il secondo lo trova occupato e finisce su quello standard. Tutti i check passano.

E qui il parcheggio saturo, con il terzo utente respinto e `parcheggioPieno` che passa a `true`.

2.3 AsmetaV – validazione con copertura (Vc)

Gli scenari li ho fatti girare con AsmetaV usando l'opzione che calcola anche la copertura, quella che la consegna chiama "Vc". La differenza rispetto alla validazione normale è che, oltre a dirti se i `check` passano, ti stampa anche quali regole del modello sono state davvero eseguite. Mettendo insieme i sei scenari scritti a mano si coprono tutte e 8 le regole di transizione, la main rule e tutti i rami di `r_gestione_VER`.

Type	Functions	State 0	State 1	State 2	State 3	State 4	State 5	State 6	State 7	State 8	State 9	State 10	State 11
C	step__	0	1	2	3	4	5	6	7	8	9	9	9
C	stato		CHK_IN	VER	INGR	IDLE	CHK_IN	VER	INGR	IDLE	IDLE	IDLE	IDLE
C	sens_in		false	false	false	true	false	false	false	false	false	false	false
C	result		1	1	1	1	1	1	1	1	1	1	1
C	utente_rilevato		DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE
C	posto_assegnato				POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD
C	transito_ok				true	false	false	false	true	true	true	true	true
C	posti_dis				0	0	0	0	0	0	0	0	0
C	posti_std									0	0	0	0

Figure 5: Animazione di test_disabile_fallback.avalla

Type	Functions	State 0	State 1	State 2	State 3	State 4	State 5	State 6	State 7	State 8	State 9	State 10	State 11	State 12	State 13	State 14	State 15
C	step__	0	1	2	3	4	5	6	7	8	9	10	11	12	13	13	13
C	stato		CHK_IN	VER	INGR	IDLE	CHK_IN	VER	INGR	IDLE	CHK_IN	VER	NEG	IDLE	IDLE	IDLE	IDLE
C	sens_in		false	false	false	true	false	false	false	true	false	false	false	false	false	false	false
C	result		1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
C	utente_rilevato		STD	STD	STD	STD	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISA
C	posto_assegnato				POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO
C	transito_ok				true	false	false	false	true	false	false	false	false	false	false	false	false
C	posti_std				0	0	0	0	0	0	0	0	0	0	0	0	0
C	posti_dis								0	0	0	0	0	0	0	0	0
C	auto_via											true	true	true	true	true	true

Figure 6: Animazione di test_pieno_totale.avalla

A quel punto mi è venuto da confrontarli con quelli generati a caso, e il risultato per me è la cosa più interessante di questa parte. Un primo export di 15 step random copriva solo 6 regole su 9: restavano fuori `r_gestione_NEG`, `r_gestione_TARIF` e `r_gestione_USC`. Ho raddoppiato a 30 step (`test_random_30steps.avalla`) e la rule coverage è arrivata al 100%, ma la branch coverage no: `r_gestione_USC` al 33%, `r_gestione_TARIF` al 50%, `r_gestione_IDLE` al 75%.

Il motivo si capisce guardando un caso concreto. Il ramo del pagamento fallito in `TARIF` vuole che il generatore metta `pagamento_ok` a false proprio mentre il sistema si trova in `TARIF`, e a caso capita raramente. Infatti quel ramo lo copre solo lo scenario scritto apposta, `test_pagamento_std`. La generazione random quindi aiuta, ma i casi limite bisogna ancora andarseli a cercare a mano.

2.4 Model checking (asmata/src/SmartParking_MC.asm)

Le CTLSPEC che ho scritto sono 10. Otto me le aspettavo vere e le ho divise in tre gruppi: safety (cose che non devono mai succedere), raggiungibilità (cose che devono poter succedere) e liveness (cose che prima o poi devono succedere). Le altre due le ho messe false apposta, per vedere il controesempio.

#	CTLSPEC	Categoria	Esito
1	<code>ag(posti_std >= 0)</code>	Safety	TRUE
2	<code>ag(posti_dis >= 0)</code>	Safety	TRUE

#	CTLSPEC	Categoria	Esito
3	ag((stato=INGR and posto_-assegnato=POSTO_STD) implies posti_-std>0)	Safety	TRUE
4	ag(posti_std <= 1)	Safety	TRUE
5	ag(posti_dis <= 1)	Safety	TRUE
6	ag((stato=INGR and posto_-assegnato=POSTO_DIS) implies posti_-dis>0)	Safety	TRUE
7	ef(stato = TARIF)	Raggiungibilità	TRUE
8	ag(stato=TARIF implies ef(stato=IDLE))	Liveness	TRUE
9	ag(stato != NEG)	Falsa (voluta)	FALSE
10	ag(stato=TARIF implies af(stato=IDLE))	Falsa (voluta)	FALSE

Table 5: Le 10 CTLSPEC sottoposte ad AsmetaSMV: 8 verificate, 2 volutamente false.

Le prime sei dicono in sostanza che i contatori non impazziscono: non vanno mai sotto zero, non vanno mai sopra la capacità, e non mando mai un’auto dentro su un tipo di posto che non ha più disponibilità. La settima dice che lo stato di pagamento è raggiungibile, l’ottava che da TARIF si riesce sempre a tornare in IDLE.

Sulle due false vale la pena spendere due parole in più.

La #9 dice “l’accesso non viene mai negato”. NuSMV la smonta con un controesempio di 9 stati: entra un abbonato che si prende l’unico posto standard, arriva un secondo veicolo, la verifica non trova posto e va in NEG. La cosa che mi ha fatto piacere è che in quella traccia `posti_dis` è ancora 1. Cioè il controesempio, oltre a dirmi che NEG è raggiungibile, mi conferma che il posto disabile non viene dato a chi non ne ha diritto nemmeno quando è l’ultima risorsa rimasta.

La #10 è la stessa cosa della #8 ma con `af` al posto di `ef`. La #8 chiede che da TARIF *esista* un cammino che torna in IDLE, la #10 che ci si torni su *tutti* i cammini. Ed è falsa, perché niente vieta all’ambiente di tenere `pagamento_ok` a false per sempre: un utente che non paga mai. Il controesempio è proprio quello, il loop infinito in TARIF. Le ho lasciate tutte e due nel modello perché messe una accanto all’altra fanno vedere bene la differenza tra i due operatori.

2.4.1 Model Advisor (opzionale)

Il Model Advisor passa anche lui per la traduzione in NuSMV, quindi sul modello principale non parte proprio: dà “Error Domain Integer not supported”. L’ho lanciato sul modello semplificato. Quello che è uscito:

- **MP1, MP3, MP4, MP7** non segnalano niente: nessun aggiornamento inconsistente, nessuna assegnazione banale, nessuna locazione da rimuovere o mai aggiornata.
- **MP2** (“conditional rule not complete”) segnala invece tutte le regole di transizione. All’inizio mi ero preoccupato, poi ho capito che sono falsi positivi: l’advisor segnala ogni `if` senza `else`, ma in ASM l’else che manca vuol dire “non aggiornare niente”, ed è proprio quello che mi serve per i self-loop. `r_gestione_NEG` deve restare in NEG finché l’auto non se ne va, e `r_gestione_TARIF` deve restare in TARIF se il pagamento non passa. Se ci mettessi un else esplicito il comportamento sarebbe identico ma il codice più lungo.

```

** Coverage Info: **
** covered rules: **
SmartParking::r_gestione_USC()
-> branch coverage:      33.33%
-> rule coverage:        75.00%
-> update rule coverage: 75.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_CHK_IN()
-> branch coverage:      - (no branches to be covered)
-> rule coverage:        100.00%
-> update rule coverage: 100.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_INGR()
-> branch coverage:      33.33%
-> rule coverage:        71.43%
-> update rule coverage: 66.67%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_VER()
-> branch coverage:      50.00%
-> rule coverage:        60.00%
-> update rule coverage: 40.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_IDLE()
-> branch coverage:      75.00%
-> rule coverage:        100.00%
-> update rule coverage: 100.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_Main()
-> branch coverage:      87.50%
-> rule coverage:        88.24%
-> update rule coverage: - (no update rules to be covered)
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_CHK_OUT()
-> branch coverage:      50.00%
-> rule coverage:        66.67%
-> update rule coverage: 50.00%
-> forall rule coverage: - (no forall rules to be covered)
** NOT covered rules: **
SmartParking::r_gestione_NEG()
SmartParking::r_gestione_TARIF()

```

Figure 7: Report di copertura AsmetaV (Vc)

- **MP5/MP6** dicono “Domain Posti could be reduced: element {2} could be removed”. Anche questo non è un difetto: il 2 l’avevo messo apposta, come ho spiegato sopra, e l’advisor mi sta solo confermando che non viene mai raggiunto. Cioè mi dà la stessa informazione delle CTLSPEC 4 e 5, per un’altra strada.

2.4.2 ATGT – generazione automatica di scenari (opzionale)

Con ATGT ho generato altri scenari in automatico, che stanno in `asmeta/src/abstracttests/`. Sono cinque file `testtest*.avalla` e si possono aprire direttamente dal repository.

Il discorso su quanto servano davvero è lo stesso che ho fatto prima per lo scenario random da 30 step, quindi non lo ripeto: si veda §2.3.

```

model checking (w execution) on /Users/francesco/Documents/INGEGNERIA/MAGISTRALE/TESTING/eclipse_asmeta_
Execution of NuSMV code ...
-----
> NuSMV -dynamic -coi -quiet SmartParking_MC.smv
-- specification AG posti_std >= 0 is true
-- specification AG posti_dis >= 0 is true
-- specification AG ((posto_assegnato = POSTO_STD & stato = INGR) -> posti_std > 0) is true
-- specification AG posti_std <= 1 is true
-- specification AG posti_dis <= 1 is true
-- specification AG ((stato = INGR & posto_assegnato = POSTO_DIS) -> posti_dis > 0) is true
-- specification EF stato = TARIF is true
-- specification AG (stato = TARIF -> EF stato = IDLE) is true
-- specification AG stato != NEG is false
-- as demonstrated by the following execution sequence
Trace Description: CTL Counterexample
Trace Type: Counterexample
-> State: 1.1 <-
  stato = IDLE
  transito_ok = false
  auto_via = false
  sens_out = false
  sens_in = false
  utente_rilevato = ABBONATO
  pagamento_ok = false
  posti_dis = 1
  posti_std = 1
  posto_assegnato = NESSUNO
-> State: 1.2 <-
  sens_in = true
-> State: 1.3 <-
  stato = CHK_IN
  sens_in = false
-> State: 1.4 <-
  stato = VER
-> State: 1.5 <-
  stato = INGR
  transito_ok = true
  posto_assegnato = POSTO_STD
-> State: 1.6 <-
  stato = IDLE
  transito_ok = false
  sens_in = true
  posti_std = 0
-> State: 1.7 <-
  stato = CHK_IN
  sens_in = false
-> State: 1.8 <-
  stato = VER
-> State: 1.9 <-
  stato = NEG
-- specification AG (stato = TARIF -> AF stato = IDLE) is false
-- as demonstrated by the following execution sequence
Trace Description: CTL Counterexample
Trace Type: Counterexample
-> State: 2.1 <-
  stato = IDLE
  transito_ok = false
  auto_via = false
  sens_out = false
  sens_in = false
  utente_rilevato = ABBONATO
  pagamento_ok = false
  posti_dis = 1
  posti_std = 1
  posto_assegnato = NESSUNO
-> State: 2.2 <-
  sens_out = true
-> State: 2.3 <-
  stato = CHK_OUT
  sens_out = false
  utente_rilevato = STD
-- Loop starts here
-> State: 2.4 <-
  stato = TARIF
  utente_rilevato = ABBONATO
-> State: 2.5 <-
model checking (w execution) finished

```

Figure 8: Esito delle 10 CTLSPEC in AsmetaSMV

Type/Functions	State 0	State 1	State 2	State 3	State 4	State 5	State 6	State 7	State 8	State 9	State 10	State 11	State 12	State 13	State 14	State 15
C stato	IDLE	CHK_OUT	USC	USC	IDLE	CHK_OUT	USC	USC	IDLE	CHK_IN	VER	INGR	INGR	IDLE	CHK_IN	VER
M sens_in	false	false	false	false	false	false	false	false	true	true	true	true	true	true	true	
M sens_out	true	true	true	true	true	true	true	true	true	true	true	true	true	true	true	
M utente_rilevato		ABBONATO	ABBONATO	ABBONATO	ABBONATO	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	STD	STD	STD	STD	STD	
M transito_ok			false	true	true	true	false	true	true	true	true	false	true	true	true	
C posto_assegnato				NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD
C posti_std											1	1	1	0	0	0
C posti_dis											1	1	1	1	1	1

Figure 9: Esecuzione random-step di verifica manuale

```

MP2: Every case rule without otherwise must be complete
NONE

MP3: Choose rule is always/sometimes/never not empty
NONE

MP3: Forall rule is always/sometimes/never not empty
NONE

MP3: Conditional rule eval to true
NONE

MP4: No assignment is always trivial
NONE

MP5: For every domain element e, there exists a location which has value e
Domain Posti could be reduced in size. Elements {2} could be removed.

MP6: Every controlled location can take any value in its codomain
Function posti_dis does not take the values {2} of its domain. It could be defined over the smaller domain {0, 1}.
Function posti_std does not take the values {2} of its domain. It could be defined over the smaller domain {0, 1}.

MP7: a location could be removed
NONE

MP7: a controlled location is never updated
NONE

MP7: a controlled location could be static
NONE

model advising finished

```

Figure 10: Output del Model Advisor (estratto)

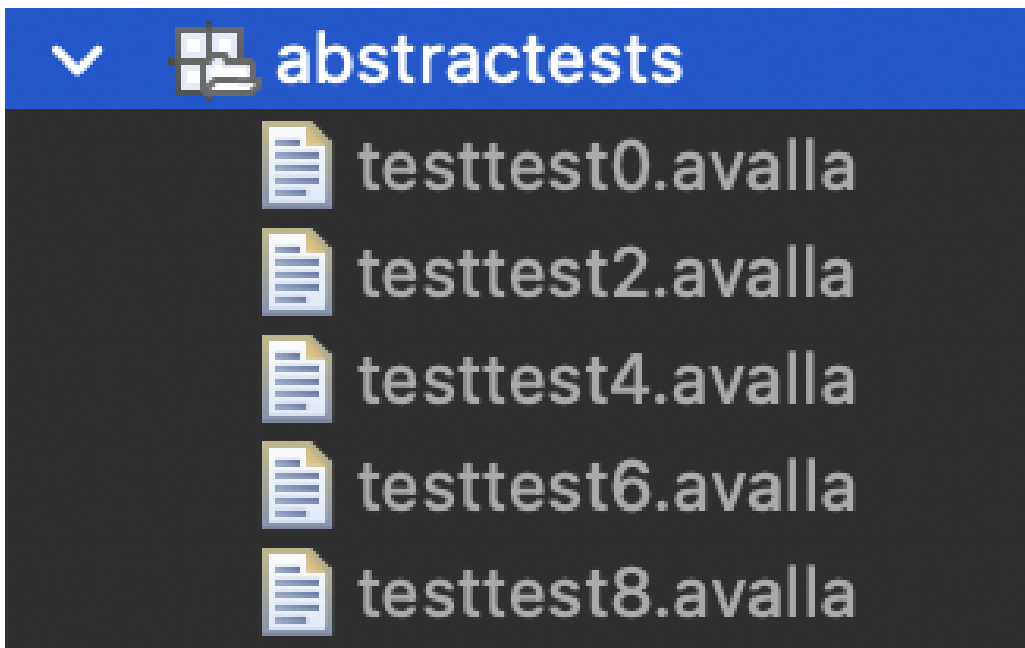


Figure 11: Elenco degli scenari generati da ATGT

3 Implementazione Java

3.1 Struttura del progetto Maven

Il progetto Java sta nella cartella `java/` ed è un progetto Maven normale, su Java 17. Le dipendenze principali sono Spring Boot con Vaadin 24 per la UI, JUnit 5 e Mockito per i test, JaCoCo per la copertura.

Il cuore sta nel package `it.tvsw.smartparking.core`, dove ho messo:

- `StatoSistema`, `TipoUtente`, `TipoPosto`: tre enum, uguali ai domini del modello ASM;
- `Parcheggio`: il nucleo, ed è l'unica classe con i contratti JML (Sezione 5);
- `SensorInput`: mette insieme i sei input monitorati di uno step;
- `Display`: un'interfaccia per le notifiche, che poi mi serve per Mockito;
- `SmartParkingFSM`: la macchina a stati vera e propria.

Fuori dal core ci sono solo la vista Vaadin e la classe di avvio di Spring Boot.

3.2 La macchina a stati

`SmartParkingFSM` ha un metodo `step(SensorInput)` che fa la stessa cosa della main rule dell'ASM: guarda in che stato siamo e chiama il gestore giusto.

Listing 4: `step()` – dispatch sullo stato corrente

```

1 public void step(SensorInput input) {
2     switch (stato) {
3         case IDLE:      gestisciIdle(input); break;
4         case CHK_IN:    gestisciChkIn(); break;
5         case VER:       gestisciVer(input); break;
6         case NEG:       gestisciNeg(input); break;
7         case INGR:      gestisciIngr(input); break;
8         case CHK_OUT:   gestisciChkOut(input); break;
9         case TARIF:     gestisciTarif(input); break;
10        case USC:       gestisciUsc(input); break;
11    }
12 }
```

Ogni `case` corrisponde a una regola dell'ASM, quindi il confronto tra i due livelli si fa a colpo d'occhio. C'è però una differenza che è meglio dire subito. Nel modello ASM il contatore viene scalato solo al transito vero e proprio, cioè in `INGR`, mentre qui la classe `Parcheggio` decide e scala tutto insieme dentro `assegnaPosto`, che viene chiamato da `gestisciVer`. Negli scenari di test del progetto la differenza non si vede, perché nessuno va a controllare lo stato in mezzo ai due passi, e l'ho fatto apposta per tenere la classe più semplice. Ne riparlo meglio nell'ispezione del codice, in Sezione 6.

3.3 Interfaccia utente (Vaadin)

Per la UI ho fatto una vista sola, `MainView`, che mostra lo stato corrente e quanti posti sono ancora liberi. I sensori li ho simulati con dei bottoni: “Arriva auto STD/DISABILE/ABBONATO”, “Transito”, “Pagamento”, “Auto va via” e “Auto in uscita”. Cliccandoli si fa avanzare la macchina a stati come se arrivassero gli input dai sensori veri.

Questa è la pagina appena aperta: stato `IDLE` e i due contatori a 1.

Qui ho cliccato “Arriva auto STD” e poi “Transito”. Siamo tornati in `IDLE` ma i posti standard sono scesi a 0.

SmartParking - Simulatore

Stato: IDLE

Posti standard liberi: 1

Posti disabili liberi: 1

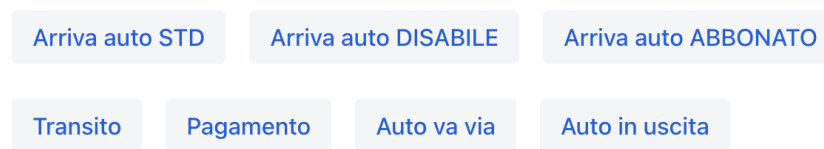


Figure 12: UI Vaadin – stato iniziale

SmartParking - Simulatore

Stato: IDLE

Posti standard liberi: 0

Posti disabili liberi: 1

Sistema pronto

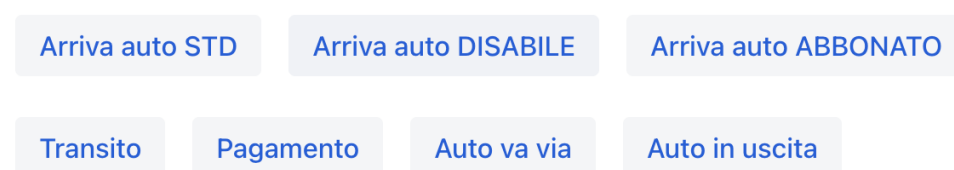


Figure 13: UI Vaadin – dopo un ingresso STD

Con il parcheggio pieno, se provo a far arrivare un'altra auto compare il messaggio di accesso negato.

E questo è lo stato TARIF: ho fatto uscire l'utente STD e il sistema si è fermato ad aspettare il pagamento.

SmartParking - Simulatore

Stato: NEG

Posti standard liberi: 0

Posti disabili liberi: 1

Accesso negato

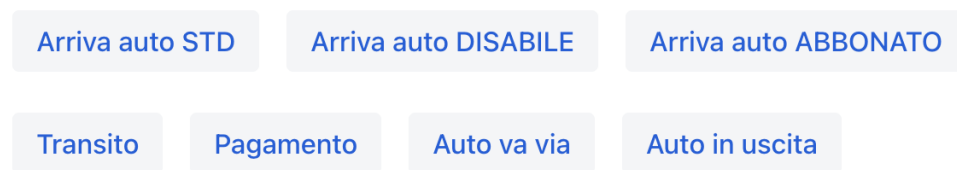


Figure 14: UI Vaadin – accesso negato a parcheggio pieno

SmartParking - Simulatore

Stato: TARIF

Posti standard liberi: 0

Posti disabili liberi: 1

In attesa di pagamento

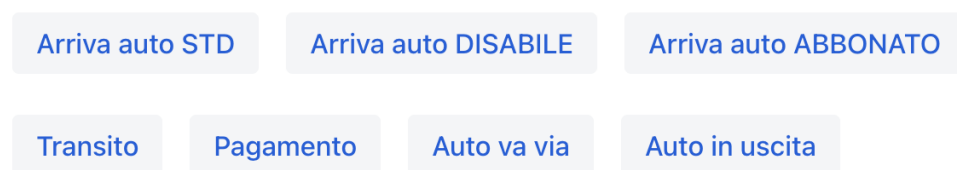


Figure 15: UI Vaadin – attesa pagamento

3.4 Test end-to-end con Selenium

La UI l'ho provata anche con un test end-to-end in Selenium (`UISeleniumTest`). L'ho taggato "ui" e tolto dalla build normale, perché per girare ha bisogno dell'applicazione già avviata e di Chrome installato, e in CI non ci sono.

Quello che fa è: apre la pagina con ChromeDriver in modalità headless, aspetta che Vaadin finisca di disegnare la pagina lato client (con `WebDriverWait`, altrimenti trova il DOM vuoto), clicca “Arriva auto STD” e controlla che lo stato mostrato diventi INGR oppure NEG. Per lanciarlo serve `mvn test -Dtest.groups=ui -Dtest.excludedGroups=`, oppure lo si fa partire da Eclipse come un normale test JUnit.

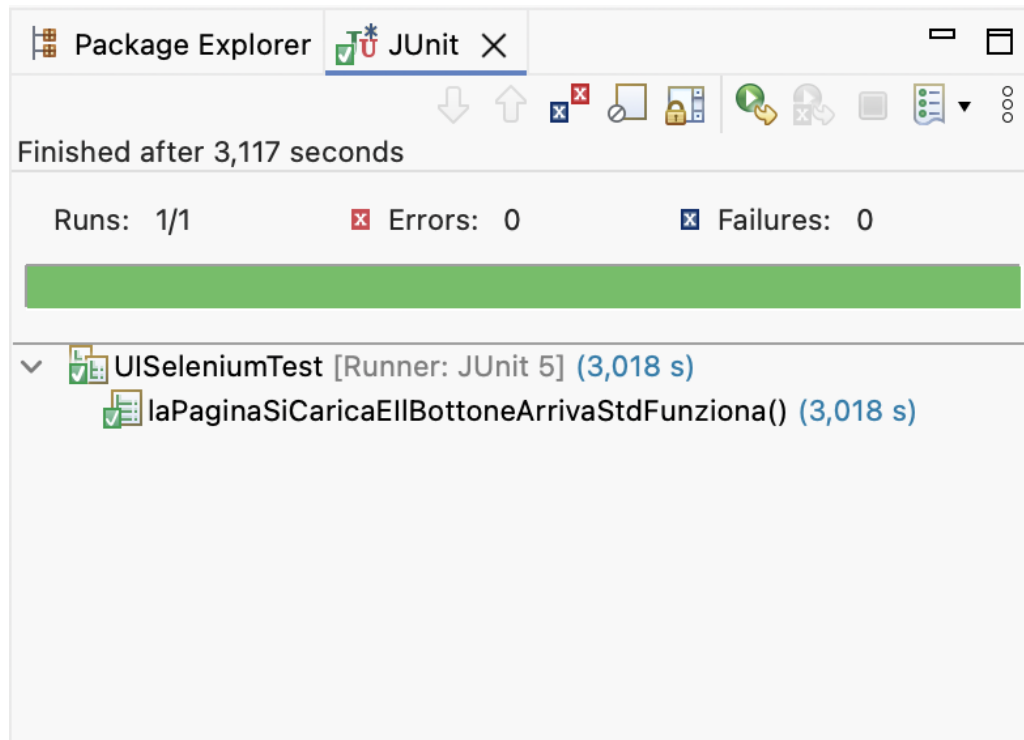


Figure 16: Test Selenium end-to-end superato

4 Testing Java

4.1 Struttura dei test

Ho diviso i test in più classi, una per ogni cosa che volevo coprire, invece di buttare tutto insieme. Così se un test si rompe si capisce subito di che tipo di problema si tratta. Le classi sono queste:

Classe	Cosa copre
<code>ParcheggioTest</code>	Costruttori, preconditioni violate, tutti i rami di <code>assegnaPosto</code> e <code>liberaPosto</code> , più un test parametrico con CSV.
<code>SmartParkingFSMTest</code>	Tutti gli 8 stati e le transizioni, il fallback del disabile, il parcheggio pieno, i self-loop di NEG, INGR e TARIF.
<code>McdcVerificaTest</code>	Copertura MC/DC delle due decisioni composte di <code>assegnaPosto</code> .
<code>DisplayMockTest</code>	Le notifiche al <code>Display</code> , controllate con Mockito.
<code>ScenarioAvallaTest</code>	Traduzione di <code>test_pieno.avalla</code> in JUnit (model based testing).
<code>CombinatorialTest</code>	La tabella pairwise a 6 righe.

Classe	Cosa copre
UISeleniumTest	Smoke test della UI, taggato <code>ui</code> e fuori dalla build normale.

4.2 MC/DC

Dentro `assegnaPosto` ci sono due decisioni composte, cioè con più di una condizione legata da and/or, e su quelle ho fatto la copertura MC/DC. La regola è che per ogni condizione serve una coppia di casi di test in cui cambia solo quella condizione e cambia anche il risultato della decisione. Vediamole una alla volta.

La prima è quella del ramo STD/ABBONATO: $D_1 = (\text{utente}=\text{STD} \vee \text{utente}=\text{ABBONATO}) \wedge \text{postiStd} > 0$. Le tre condizioni le chiamo A, B e C nell'ordine in cui compaiono.

Caso	A	B	C	D_1	Esito	Coppia indipendente
TC1	T	F	T	T	POSTO_STD	base
TC2	T	F	F	F	NESSUNO	isola C (vs TC1)
TC3	F	F	T	F	POSTO_DIS	isola A (vs TC1)
TC4	F	T	T	T	POSTO_STD	isola B (vs TC3)

Table 7: MC/DC – Decisione 1.

Leggendola: TC1 è il caso base, un utente STD con un posto libero. TC2 cambia solo C (niente posti) e la decisione diventa falsa, quindi la coppia TC1–TC2 isola C. TC3 cambia solo A rispetto a TC1 ed è la coppia per A. TC4 confrontato con TC3 cambia solo B ed è la coppia per B. Con quattro casi sono coperte tutte e tre le condizioni.

La seconda decisione è quella del ramo DISABILE, cioè il fallback, con $C_1 = \text{postiDis} > 0$ e $C_2 = \text{postiStd} > 0$.

Caso	C_1	C_2	Esito	Coppia indipendente
TC5	T	—	POSTO_DIS	base
TC6	F	T	POSTO_STD	isola C_1 (vs TC5)
TC7	F	F	NESSUNO	isola C_2 (vs TC6)

Table 8: MC/DC – Decisione 2.

Qui in TC5 il valore di C_2 non conta, perché se c'è il posto disabile il secondo if non viene nemmeno valutato: per questo ho messo un trattino.

4.3 Combinatorial testing (pairwise)

L'assegnazione del posto dipende da tre parametri: il tipo di utente (tre valori) e i due contatori, che nel modello valgono 0 oppure 1. Tutte le combinazioni sarebbero $3 \times 2 \times 2 = 12$, che sono poche e si potrebbero anche fare tutte, ma il punto dell'esercizio era usare il pairwise. Basta quindi coprire tutte le *coppie* di valori, e per farlo bastano 6 righe invece di 12.

#	tipoUtente	postiStd	postiDis	Esito atteso
1	STD	0	0	NESSUNO
2	STD	1	1	POSTO_STD
3	DISABILE	0	1	POSTO_DIS

#	tipoUtente	postiStd	postiDis	Esito atteso
4	DISABILE	1	0	POSTO_STD (fallback)
5	ABBONATO	0	1	NESSUNO
6	ABBONATO	1	0	POSTO_STD

Table 9: Tabella pairwise (copre tutte le coppie dei 3 parametri).

Le righe 3 e 5 sono quelle che mi piacciono di più, perché messe vicine fanno vedere la regola del posto riservato: stessa configurazione di contatori, ma il disabile entra e l'abbonato no.

4.4 Mockito

La FSM, ogni volta che cambia stato, manda un messaggio a un oggetto **Display**. Per controllare che questi messaggi partano davvero senza tirare in ballo la UI, ho usato Mockito: mi creo un **Display** finto e poi con **verify** controllo che il metodo **mostra** sia stato chiamato con il testo giusto.

Listing 5: Verifica delle notifiche con Mockito

```

1 @Test
2 void notificaAccessoNegatoQuandoParcheggioPieno() {
3     SmartParkingFSM fsm = new SmartParkingFSM(new Parcheggio(0, 0), display);
4     fsm.step(SensorInput.sensIn());
5     fsm.step(SensorInput.utente(TipoUtente.STD));
6     fsm.step(SensorInput.utente(TipoUtente.STD));
7     verify(display).mostra("Accesso negato");
8 }

```

In questo test parto da un parcheggio già pieno, faccio arrivare un'auto e controllo che al display arrivi davvero "Accesso negato".

4.5 Model Based Testing

Qui l'idea è prendere uno scenario già scritto sul modello astratto e riportarlo tale e quale sul codice vero. Ho preso **test_pieno.avalla** e l'ho tradotto passo passo in **ScenarioAvallaTest**: ogni **set** dell'avalla diventa la preparazione dell'input, ogni **step** una chiamata a **step()** e ogni **check** un assert. Accanto a ciascuna riga ho lasciato un commento con la riga originale dello scenario, così il confronto tra i due file si fa senza doverli tenere aperti tutti e due.

4.6 Esecuzione e copertura

Tutti i test del package **core** passano: sono 59, più quello Selenium che sta fuori dalla build normale.

La copertura l'ho misurata con JaCoCo lanciando **mvn verify** e guardando il report in **target/site/jacoco/index.html**. Dal conteggio ho tolto due cose: la UI Vaadin, che provo end-to-end con Selenium e che JaCoCo non riesce a misurare, e la classe di avvio di Spring Boot, che sono tre righe di boilerplate. L'esclusione è scritta nel **pom.xml**.

Alla prima misurazione il package **core** stava al 98% di instruction e al 95% di branch. **Parcheggio** era già al 100% su tutte e due, il che tornava con il fatto che i suoi contratti erano già stati dimostrati con ESC. In **SmartParkingFSM** invece restavano fuori tre branch, e vale la pena guardarli uno per uno perché non sono tutti uguali:

- il self-loop di **gestisciUsc**, cioè il caso in cui in USC il transito non è ancora avvenuto;
- il ramo **display == null** di **cambiaStato**;

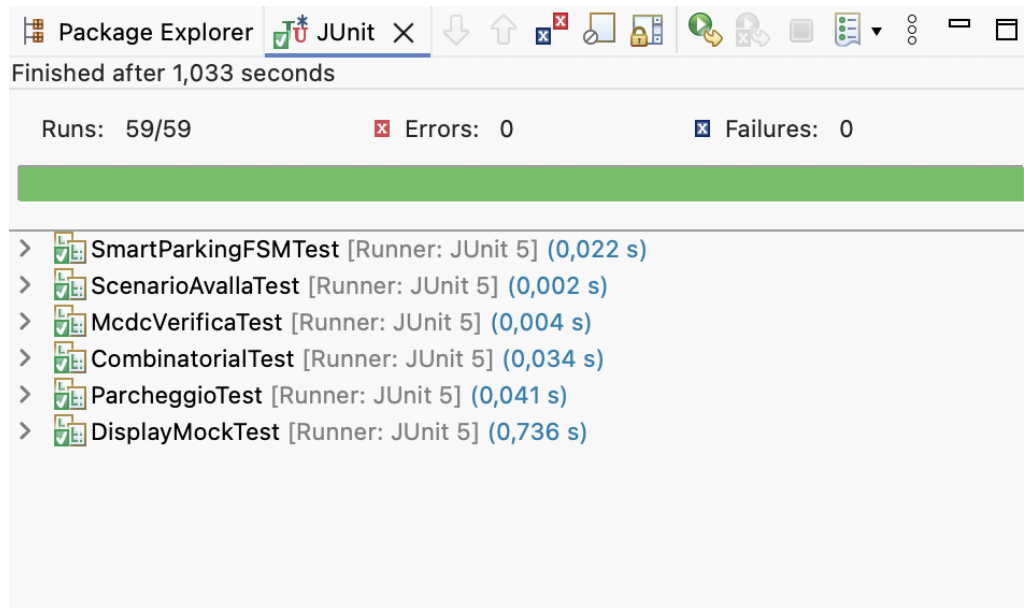


Figure 17: Esecuzione di tutta la suite JUnit

- il `default` dello switch di `step`.

I primi due erano buchi veri, mi ero semplicemente dimenticato di provare quei casi, e li ho chiusi con due test in più (`uscRestaInUscFinoAlTransito` e `fsmFunzionaSenzaDisplay`). Il terzo invece non si può coprire: lo switch gestisce tutti e 8 i valori dell'enum, quindi in quel `default` non ci si arriva mai. L'ho lasciato lo stesso perché lancia un'eccezione ed è una rete di sicurezza se un giorno qualcuno aggiunge uno stato nuovo. È lo stesso tipo di ramo infattibile che avevo già incontrato con `AsmetaV` sul modello (§2.3).

Dopo i due test aggiunti, quel `default` è rimasto l'unico branch scoperto di tutto il package.

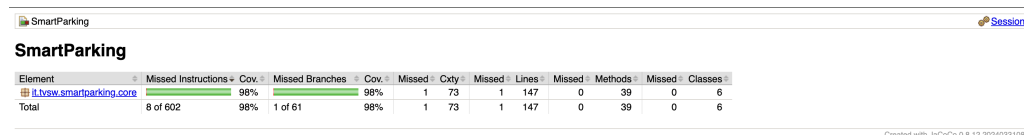


Figure 18: Report JaCoCo – totale (solo package core)

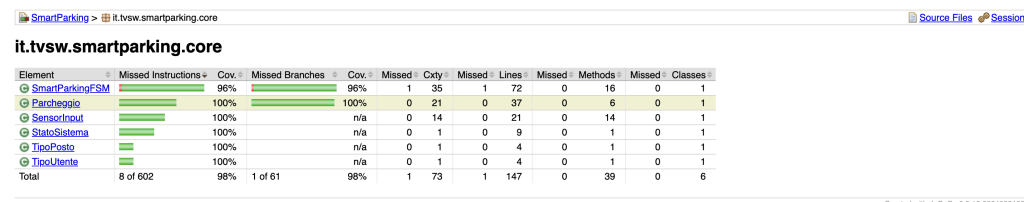


Figure 19: Report JaCoCo – dettaglio per classe del package core

5 JML e verifica statica (ESC)

Per la parte di design by contract ho annotato con JML una classe sola, `Parcheggio`. È il nucleo del dominio: tiene i due contatori dei posti liberi e gestisce l'assegnazione e il rilascio, cioè fa

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Ctx	Missed	Lines	Missed	Methods
start(SensorInput)	94%	90%	1	10	1	21	0	1		
SmartParkingFSM(Parcheggio_Display)	100%	100%	0	2	0	8	0	1		
gestisciVer(SensorInput)	100%	100%	0	2	0	7	0	1		
gestisciIdle(SensorInput)	100%	100%	0	3	0	5	0	1		
gestisciUsco(SensorInput)	100%	100%	0	2	0	5	0	1		
gestisciChkOut(SensorInput)	100%	100%	0	2	0	5	0	1		
cambiaStato(StatoSistema_String)	100%	100%	0	2	0	4	0	1		
gestisciNeg(SensorInput)	100%	100%	0	2	0	3	0	1		
gestisciIngr(SensorInput)	100%	100%	0	2	0	3	0	1		
gestisciTanti(SensorInput)	100%	100%	0	2	0	3	0	1		
SmartParkingFSM(Display)	100%	n/a	0	1	0	2	0	1		
gestisciChkIn()	100%	n/a	0	1	0	2	0	1		
getPostiStd()	100%	n/a	0	1	0	1	0	1		
getPostiDis()	100%	n/a	0	1	0	1	0	1		
getStato()	100%	n/a	0	1	0	1	0	1		
getPostoAssegnato()	100%	n/a	0	1	0	1	0	1		
Total	8 of 204	96%	1 of 31	96%	1	35	1	72	0	16

Figure 20: Report JaCoCo – dettaglio per metodo di SmartParkingFSM

quello che nel modello ASM fanno le regole `r_gestione_VER`, `r_gestione INGR` e `r_gestione_USC`.

L'ho tenuta piccola apposta e senza dipendenze da niente: dentro ci sono solo `int` ed `enum`. Il motivo è pratico, cioè che così OpenJML riesce a dimostrarla tutta, mentre su una classe che si tira dietro mezzo Spring non ci sarei mai arrivato. Tutto il resto del sistema (la FSM, la UI, il display, i sensori) è Java normale senza annotazioni, come chiedeva la consegna.

Una cosa che ho scoperto sbattendoci contro: i campi `postiStd` e `postiDis` li ho dovuti dichiarare `private /*@ spec_public @*/`. Le clausole di specifica `public`, come le invarianti e le postcondizioni, possono nominare solo roba visibile a livello `public`, e se lascio i campi semplicemente `private` OpenJML si ferma con un errore di visibilità. Con `spec_public` restano `private` per il codice ma diventano visibili alla specifica.

5.1 Invarianti

Le invarianti sono due e devono valere sempre, da dopo il costruttore in poi:

- i posti standard liberi stanno sempre tra 0 e `MAX_STD`;
- i posti disabili liberi stanno sempre tra 0 e `MAX_DIS`.

Listing 6: Invarianti di classe

```
//@ public invariant 0 <= postiStd && postiStd <= MAX_STD;
private /*@ spec_public @*/ int postiStd;

//@ public invariant 0 <= postiDis && postiDis <= MAX_DIS;
private /*@ spec_public @*/ int postiDis;
```

Sono le stesse identiche proprietà che avevo già verificato con il model checking sul modello, cioè `ag(posti >= 0)` e `ag(posti <= 1)`. Le ho scritte apposta uguali: così la stessa cosa finisce garantita a tre livelli diversi, nel modello con il model checker, nel codice con il contratto, e ancora nei test con JaCoCo.

5.2 Costruttori

I costruttori sono due. Quello di default garantisce lo stato iniziale del modello ASM, cioè tutti i posti liberi. Quello con i parametri lo uso nei test, quando mi serve partire da una configurazione qualunque, e chiede che i valori passati siano già dentro i limiti dell'invariante.

Listing 7: Contratti dei costruttori

```
//@ ensures postiStd == MAX_STD && postiDis == MAX_DIS;
```

```
public Parcheggio() { ... }

/*@ requires 0 <= postiStdIniziali && postiStdIniziali <= MAX_STD;
   @ requires 0 <= postiDisIniziali && postiDisIniziali <= MAX_DIS;
   @ ensures postiStd == postiStdIniziali && postiDis == postiDisIniziali;
public Parcheggio(int postiStdIniziali, int postiDisIniziali) { ... }
```

Il `requires` del secondo costruttore dice cosa deve garantire chi chiama, ma non protegge a runtime: se qualcuno passa un valore fuori range senza aver fatto la verifica statica, il contratto non lo ferma. Per questo dentro ho messo anche un controllo esplicito che lancia `IllegalArgumentException`, e quel comportamento lo provo in `ParcheggioTest` con `assertThrows`.

5.3 Metodo assegnaPosto

Questo è quello con il contratto più lungo, perché deve dire tutto quello che fa `r_gestione_VER`. Le postcondizioni sono due, una per ramo.

Per STD e ABBONATO ci sono due esiti possibili: o c'era un posto standard libero, e allora torna `POSTO_STD` con il contatore sceso di 1, oppure non c'era e torna `NESSUNO` senza toccare niente.

Per il DISABILE gli esiti sono tre, perché in mezzo c'è il fallback: posto disabili se disponibile, altrimenti posto standard se disponibile, altrimenti `NESSUNO`.

Listing 8: Contratto di `assegnaPosto`

```
/*@ requires utente != null;
   @ ensures (utente == TipoUtente.STD || utente == TipoUtente.ABBONATO) ==>
   @      ((\old(postiStd) > 0 && \result == TipoPosto.POSTO_STD
   @      && postiStd == \old(postiStd) - 1 && postiDis == \old(postiDis))
   @      || (\old(postiStd) == 0 && \result == TipoPosto.NESSUNO
   @      && postiStd == \old(postiStd) && postiDis == \old(postiDis)));
   @ ensures utente == TipoUtente.DISABILE ==>
   @      ((\old(postiDis) > 0 && \result == TipoPosto.POSTO_DIS
   @      && postiDis == \old(postiDis) - 1 && postiStd == \old(postiStd))
   @      || (\old(postiDis) == 0 && \old(postiStd) > 0
   @      && \result == TipoPosto.POSTO_STD
   @      && postiStd == \old(postiStd) - 1 && postiDis == \old(postiDis))
   @      || (\old(postiDis) == 0 && \old(postiStd) == 0
   @      && \result == TipoPosto.NESSUNO
   @      && postiStd == \old(postiStd) && postiDis == \old(postiDis)));
public TipoPosto assegnaPosto(TipoUtente utente) { ... }
```

Il `\old` serve per parlare del valore che il contatore aveva prima della chiamata, altrimenti non potrei dire "è sceso di uno".

Una cosa che all'inizio non avevo messo: in ogni esito ho anche scritto che l'*altro* contatore resta com'era. Senza quel pezzo il contratto sarebbe stato sottospecificato, cioè un'implementazione che assegna il posto giusto ma nel frattempo tocca anche l'altro contatore lo rispetterebbe lo stesso.

5.4 Metodo liberaPosto

Questo corrisponde al rilascio del posto in `r_gestione_USC`. Chiede che il posto non sia null e che il contatore corrispondente non sia già al massimo, e garantisce che venga incrementato di 1 solo quello giusto. Con `NESSUNO` non fa niente e lo stato resta identico.

Listing 9: Contratto di `liberaPosto`

```
/*@ requires posto != null;
   @ requires posto == TipoPosto.POSTO_STD ==> postiStd < MAX_STD;
   @ requires posto == TipoPosto.POSTO_DIS ==> postiDis < MAX_DIS;
   @ ensures posto == TipoPosto.POSTO_STD ==>
```

```
//@   postiStd == \old(postiStd) + 1 && postiDis == \old(postiDis);
//@ ensures posto == TipoPosto.POSTO_DIS ==>
//@   postiDis == \old(postiDis) + 1 && postiStd == \old(postiStd);
//@ ensures posto == TipoPosto.NESSUNO ==>
//@   postiStd == \old(postiStd) && postiDis == \old(postiDis);
public void liberaPosto(TipoPosto posto) { ... }
```

Quel secondo e terzo **requires** non c'erano nella prima versione e sono il motivo per cui la prova non chiudeva: ne parlo tra poco.

5.5 Metodi getter

I due getter li ho annotati `/*@ pure @*/`, che vuol dire senza effetti collaterali. Serve perché un metodo non puro dentro una clausola JML non si può usare. La postcondizione dice solo che restituiscono il campo, niente di più.

Listing 10: Getter puri

```
//@ ensures \result == postiStd;
public /*@ pure @*/ int getPostiStd() { ... }

//@ ensures \result == postiDis;
public /*@ pure @*/ int getPostiDis() { ... }
```

5.6 Verifica dei contratti con OpenJML (ESC)

Scritti i contratti, li ho verificati con OpenJML (release 21.0.27, build nativa per macOS arm64) in modalità `-esc`. Quello che fa il tool è tradurre codice e contratti in formule logiche e passarle al solver Z3, che deve dimostrare che *qualunque* esecuzione rispetta postcondizioni e invarianti. È una cosa diversa dai test: i test provano alcuni casi, qui invece o si dimostra per tutti o non si dimostra.

Il comando, dalla root del progetto, è questo:

Listing 11: Comando di verifica ESC

```
$ openjml --esc --progress \
  -cp java/src/main/java \
  java/src/main/java/it/tvsw/smartparking/core/Parcheggio.java
```

Un contratto che all'inizio non chiudeva

Arrivare a “tutti Verified” non è stato immediato, e il caso di `liberaPosto` secondo me è quello che spiega meglio come funziona la cosa.

La prima versione che avevo scritto era questa, con la postcondizione che descrive l'incremento ma senza il **requires** sulla capacità:

Listing 12: Prima versione di liberaPosto (senza il requires di capacità)

```
//@ requires posto != null;
//@ ensures posto == TipoPosto.POSTO_STD ==>
//@   postiStd == \old(postiStd) + 1 && postiDis == \old(postiDis);
public void liberaPosto(TipoPosto posto) { ... }
```

Il solver, non avendo nessun vincolo sul valore di partenza, prova anche il caso `postiStd == MAX_STD`. Lì `postiStd + 1` fa `MAX_STD + 1`, che viola l'invariante di classe. Quindi non riesce a ristabilire l'invariante all'uscita del metodo e mi risponde così:

Listing 13: Errore ESC sulla prima versione

```
Parcheggio.java:.. warning: The prover cannot establish an assertion
(InvariantExit) in method liberaPosto
postiStd == \old(postiStd) + 1 ...
Associated declaration: invariant 0 <= postiStd && postiStd <= MAX_STD
```

La soluzione è rafforzare la preconditione, in modo che sia chi chiama a garantire che c'è spazio per incrementare:

Listing 14: Versione corretta: il requires di capacità chiude la prova

```
//@ requires posto == TipoPosto.POSTO_STD ==> postiStd < MAX_STD;
//@ requires posto == TipoPosto.POSTO_DIS ==> postiDis < MAX_DIS;
```

Con `postiStd < MAX_STD` assunto all'ingresso, dopo l'incremento vale `postiStd <= MAX_STD`, l'invariante regge e il metodo passa a Verified.

Il senso della cosa, secondo me, è questo: ogni volta che c'è un'operazione aritmetica bisogna aver vincolato prima tutto quello che ci entra, con un `requires` o con un'invariante. `Parcheggio` chiude tutto perché ha due soli interi e tutti e due sono limitati; su un nucleo con array o contatori liberi la stessa cosa diventa molto più difficile. L'altro ostacolo che ho incontrato era di tutt'altro tipo, cioè la visibilità dei campi private, e l'ho risolto con `spec_public` come ho detto all'inizio.

```
/Users/francesco/Documents/INGEGNERIA/MAGISTRALE/TESTING/openjml-macos-arm64-21.0.27/openjml \
--esc --progress \
--cp java/src/main/java \
java/src/main/java/it/tvsw/smartparking/core/Parcheggio.java
Proving methods in it.tvsw.smartparking.core.Parcheggio
Starting proof of it.tvsw.smartparking.core.Parcheggio.Parcheggio() with prover z3-4.3.X
[Method assertions are validated [1,14 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.Parcheggio() with prover z3-4.3.X - no warnings [1,26 secs]
Starting proof of it.tvsw.smartparking.core.Parcheggio.Parcheggio(int,int) with prover z3-4.3.X
[Method assertions are validated [1,21 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.Parcheggio(int,int) with prover z3-4.3.X - no warnings [1,30 secs]
Starting proof of it.tvsw.smartparking.core.Parcheggio.assegnaPosto(it.tvsw.smartparking.core.TipoUtente) with prover z3-4.3.X
[Method assertions are validated [1,29 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.assegnaPosto(it.tvsw.smartparking.core.TipoUtente) with prover z3-4.3.X - no warnings [1,30 secs]
Starting proof of it.tvsw.smartparking.core.Parcheggio.getPostiDis() with prover z3-4.3.X
[Method assertions are validated [1,03 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.getPostiDis() with prover z3-4.3.X - no warnings [1,04 secs]
Starting proof of it.tvsw.smartparking.core.Parcheggio.getPostiStd() with prover z3-4.3.X
[Method assertions are validated [1,05 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.getPostiStd() with prover z3-4.3.X - no warnings [1,05 secs]
Starting proof of it.tvsw.smartparking.core.Parcheggio.liberaPosto(it.tvsw.smartparking.core.TipoPosto) with prover z3-4.3.X
[Method assertions are validated [1,19 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.liberaPosto(it.tvsw.smartparking.core.TipoPosto) with prover z3-4.3.X - no warnings [1,22 secs]
Completed proving methods in it.tvsw.smartparking.core.Parcheggio [7,18 secs]
Proving methods in it.tvsw.smartparking.core.TipoUtente
Starting proof of it.tvsw.smartparking.core.TipoUtente.TipoUtente() with prover z3-4.3.X
[Method assertions are validated [1,06 secs]
Completed proof of it.tvsw.smartparking.core.TipoUtente.TipoUtente() with prover z3-4.3.X - no warnings [1,07 secs]
Completed proving methods in it.tvsw.smartparking.core.TipoUtente [1,07 secs]
Proving methods in it.tvsw.smartparking.core.TipoPosto
Starting proof of it.tvsw.smartparking.core.TipoPosto.TipoPosto() with prover z3-4.3.X
[Method assertions are validated [1,06 secs]
Completed proof of it.tvsw.smartparking.core.TipoPosto.TipoPosto() with prover z3-4.3.X - no warnings [1,07 secs]
Completed proving methods in it.tvsw.smartparking.core.TipoPosto [1,07 secs]
Summary:
Valid: 8
Invalid: 0
Infeasible: 0
Timeout: 0
Error: 0
Skipped: 0
TOTAL METHODS: 8
Classes: 3 proved of 3
DURATION: 9,6 secs
(base) francesco@iPhone Progetto_Testing %
```

Figure 21: Output di openjml -esc: tutti i metodi Verified

Alla fine tutti e sei i metodi (i due costruttori, `assegnaPosto`, `liberaPosto` e i due getter) risultano Verified.

6 Analisi statica e ispezione del codice con LLM

La consegna chiedeva di fare l'ispezione "con chatgpt o LLM in generale", quindi ho usato ChatGPT come se fosse un collega a cui far dare una seconda occhiata al codice. Gli ho passato tutto il sorgente dei package `core` e `ui` e gli ho chiesto di leggerlo cercando nomi poco chiari, casi limite non gestiti, possibili `NullPointerException` e punti in cui il codice si allontana dal modello ASM.

Le cose che sono uscite non le ho prese per buone così com'erano. Le ho valutate una per una e per ognuna ho deciso se correggerla, mitigarla o lasciarla stare spiegando perché. In pratica l'LLM mi ha segnalato i punti su cui ragionare, ma le decisioni le ho prese io.

6.1 Checklist preliminare

Prima di guardare le segnalazioni ho fatto un giro di controlli di base:

- ✓ La funzionalità descritta nei requisiti c'è tutta nel codice.
- ✓ Le variabili sono inizializzate prima dell'uso, e i costruttori di `Parcheggio` validano i parametri.
- ✓ Gli enum sono confrontati con `==`, che per gli enum Java è corretto.
- ✓ Lo `switch` in `step` ha un ramo `default` che lancia eccezione se arriva uno stato non gestito.
- ✓ Le eccezioni di preconditione sono usate in modo coerente in tutta la classe.

6.2 Le osservazioni e come le ho gestite

1. **Manca il controllo di null su `utenteRilevato` in `CHK_OUT`:** il confronto restituisce false senza dire niente, e il sistema va in USC come se l'utente non fosse STD. L'ho lasciata così per restare fedele all'ASM, dove il dominio `TipoUtente` non prevede l'assenza del dato; in più tutti i punti che chiamano quel metodo impostano sempre l'utente.
2. **`TipoPosto.NESSUNO` vuol dire due cose diverse:** in `assegnaPosto` significa “non c'è posto”, in `liberaPosto` significa “non fare niente”. Ho chiarito la cosa nel Javadoc invece di inventarmi un tipo separato, che avrebbe appesantito una classe che volevo tenere semplice.
3. **`liberaPosto` lancia eccezione se il contatore è già al massimo.** Questa l'ho tenuta apposta: preferisco un crash che si vede subito a un contatore che va oltre la capacità senza dire niente, violando l'invariante. Il caso è comunque coperto da un test.
4. **Il decremento avviene in un momento diverso rispetto all'ASM:** qui `assegnaPosto` decide e scala insieme, nell'ASM si scala al transito. Negli scenari di test la differenza non si vede, quindi ho tenuto la versione più semplice e l'ho scritta nel Javadoc della classe.
5. **Non c'è thread-safety.** Va bene così, perché c'è un varco solo pilotato da una FSM sequenziale, esattamente come nel modello. L'ho segnata come limite se un giorno si volessero gestire più varchi.
6. **`posto_assegnato` è una memoria da un posto solo, e il difetto c'è anche nel modello ASM.** Questa è l'osservazione più importante di tutte. Sia la FSM Java sia l'ASM si ricordano un solo posto assegnato, ma nel parcheggio ci stanno due veicoli, uno standard e uno disabili. Se entra uno STD, poi entra un DISABILE che sovrascrive lo slot, e poi esce il primo, viene rilasciato il posto sbagliato e i contatori si sballano.

Il punto è che l'implementazione è fedele alla specifica: è la specifica stessa che dà per scontato di tracciare un veicolo alla volta, senza mai associare un veicolo al suo posto. Per sistemarlo servirebbe una mappa veicolo→posto sia nell'ASM sia in Java. L'ho lasciato e documentato come limite noto, perché mi sembra l'esempio più utile di tutto il progetto: è un difetto che il testing di conformità non può trovare, visto che il codice fa esattamente quello che dice il modello, e che salta fuori solo leggendo il codice con occhio critico.

7. **I bottoni della UI sono sempre attivi:** `MainView` non li disabilita in base allo stato, quindi si può cliccare “Pagamento” mentre siamo in `IDLE`. Non succede niente di male perché quegli step finiscono assorbiti dai self-loop della FSM, cioè proprio dagli `if` senza `else` che il Model Advisor segnalava come MP2 (§2.3). Per un prototipo va bene; in una versione vera si metterebbe un `setEnabled` legato allo stato.

6.3 Bilancio

In tutto sono uscite 7 osservazioni. Nessun bug vero da correggere nel codice, e in due casi mi è bastato chiarire le cose nel Javadoc. È però saltato fuori un difetto nella specifica, quello del punto 6, che né il testing di conformità né la verifica ESC riuscivano a vedere: i contratti JML di `Parcheggio` restano rispettati anche nell’esecuzione che rilascia il posto sbagliato, perché il problema sta più in alto, a livello di FSM e di modello.

Per me è la conferma che questi livelli di verifica servono a cose diverse. Il model checking e l’ESC dimostrano quello che è stato formalizzato; l’ispezione a mano trova quello che nella formalizzazione non c’è.

7 Continuous Integration

La continuous integration l’ho messa su con GitHub Actions. Il senso è che a ogni push o pull request i test partono da soli, così se rompo qualcosa me ne accorgo subito invece che due settimane dopo.

Tutto sta nel file `.github/workflows/ci.yml`: fa la build Maven completa, lascia fuori i test taggati `ui` (che hanno bisogno di Chrome e dell’app avviata, e sul runner non ci sono) e alla fine salva il report JaCoCo come artifact scaricabile.

Listing 15: `.github/workflows/ci.yml` (estratto)

```

1 on:
2   push:
3   pull_request:
4
5 jobs:
6   build-and-test:
7     runs-on: ubuntu-latest
8     steps:
9       - uses: actions/checkout@v4
10      - uses: actions/setup-java@v4
11        with:
12          distribution: temurin
13          java-version: '17'
14      - run: mvn -B -f java/pom.xml verify
15      - uses: actions/upload-artifact@v4
16        with:
17          name: jacoco-report
18          path: java/target/site/jacoco/

```

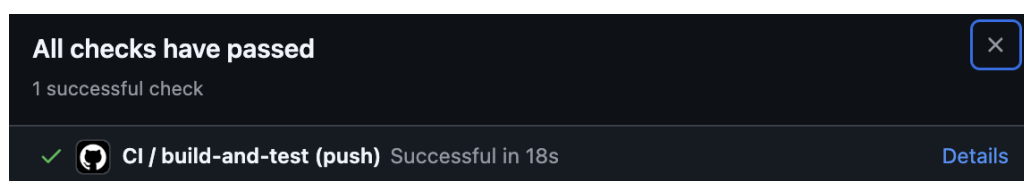


Figure 22: Esecuzione CI riuscita sul branch `main`

Questo è il run verde su `main`.

Solo che avere la pipeline verde, di per sé, dimostra poco: dimostra che i test passano, non che la CI se ne accorgerebbe se smettessero di passare. Per essere sicuro che il meccanismo funzionasse ho quindi rotto il codice apposta. Su un branch a parte, che ho chiamato `ci_failure` e che ho lasciato sul repository come prova, ho modificato `assegnaPosto` facendogli restituire sempre `TipoPosto.NESSUNO`. Al push la pipeline è andata in rosso, con i test di `ParcheggioTest` e `SmartParkingFSMTest` falliti sugli assert dell'assegnazione. Il branch `main` è rimasto com'era.

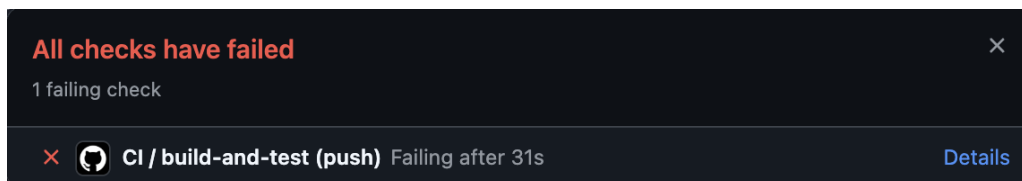


Figure 23: Esecuzione CI fallita sul branch `ci_failure` (regressione introdotta di proposito)

Quello che mi porto a casa da questa parte è che il valore della CI non è tanto far girare i test, che posso farlo anche in locale, quanto non potermene dimenticare: ogni commit resta legato a una prova che in quel momento il codice funzionava.

8 Conclusioni

Il progetto copre tutte e quattro le parti chieste dalla consegna: i requisiti, il modello Asmeta con gli scenari e il model checking, i contratti JML sul nucleo e l'implementazione Java con i test, la UI, la CI e l'analisi statica.

8.1 Riepilogo delle evidenze

- Simulatore e animatore Asmeta (Sezione 2)
- AsmetaV – copertura degli scenari Avalla (Sezione 2)
- AsmetaSMV – 8 CTLSPEC verificate e 2 volutamente false con controesempio (Sezione 2)
- Model Advisor e ATGT, opzionali (Sezione 2)
- OpenJML ESC sui contratti del nucleo (Sezione 3)
- UI Vaadin in esecuzione e test Selenium (Sezione 4)
- JUnit (59 test del package `core`, tutti verdi, più 1 test Selenium escluso dalla build standard) e copertura JaCoCo (Sezione 5)
- Analisi statica e ispezione critica con LLM (Sezione 6)
- GitHub Actions: build e test automatici a ogni push, più il branch `ci_failure` come prova che la pipeline intercetta le regressioni (Sezione 7)

8.2 Cosa mi porto a casa

La cosa che mi è rimasta di più, facendolo, è che le varie tecniche non si sovrappongono ma guardano posti diversi.

Il model checking mi ha detto che i contatori non possono impazzire in nessuno stato raggiungibile, cosa che con i test non avrei mai potuto dimostrare, al massimo controllare in qualche caso. L'ESC mi ha dato la stessa garanzia sul codice vero, però solo su una classe che ho dovuto tenere apposta piccola e senza dipendenze. I test coprono anche il resto, ma solo

nei casi che mi sono venuti in mente, e infatti la coverage mi ha fatto scoprire due rami che mi ero dimenticato. L'ispezione a mano, per ultima, ha trovato l'unica cosa che tutte le altre si erano perse, cioè il difetto della memoria a slot singolo: il codice era conforme al modello, quindi nessuna verifica automatica poteva segnalarlo, perché il problema stava nel modello.

Se dovessi rifarlo, la cosa che cambierei è proprio quella: mettere l'associazione veicolo→posto già nell'ASM, invece di scoprirne la mancanza alla fine.